

Especificacao Tecnica

Regulatory Compliance Engine — Especificação Técnica

| Motor de conformidade regulatória para tropicalização de serviços financeiros (EUA → Brasil)

Nota de originalidade: projeto **fictício e original**, desenvolvido do zero para fins de portfólio. Cenário, dados e regras são ilustrativos; não reproduz sistemas, código, arquitetura ou soluções proprietárias de nenhuma empresa. As tecnologias citadas (Go, Fiber, MongoDB, PHP) são de domínio público.

1. A ideia em uma frase

Assim como um produto importado passa pela **alfândega** antes de entrar no país, um **produto financeiro criado sob as regras dos EUA** precisa passar por uma verificação de conformidade antes de operar no Brasil. Este projeto é essa "alfândega regulatória": recebe uma operação ou a configuração de um produto e responde "**pode operar no Brasil? o que precisa ser adaptado?**", aplicando regras regulatórias brasileiras de forma configurável.

2. Contexto e problema

Empresas de tecnologia que atendem o setor financeiro convivem com um desafio recorrente: **traduzir norma regulatória em software**. Quando um serviço financeiro concebido no exterior (tipicamente nos EUA) é trazido para o Brasil — o processo de *tropicalização* —, ele precisa se adequar a exigências locais que não existiam na origem: tributação específica, limites, obrigações de comunicação a órgãos reguladores, prevenção à lavagem de dinheiro e proteção de dados.

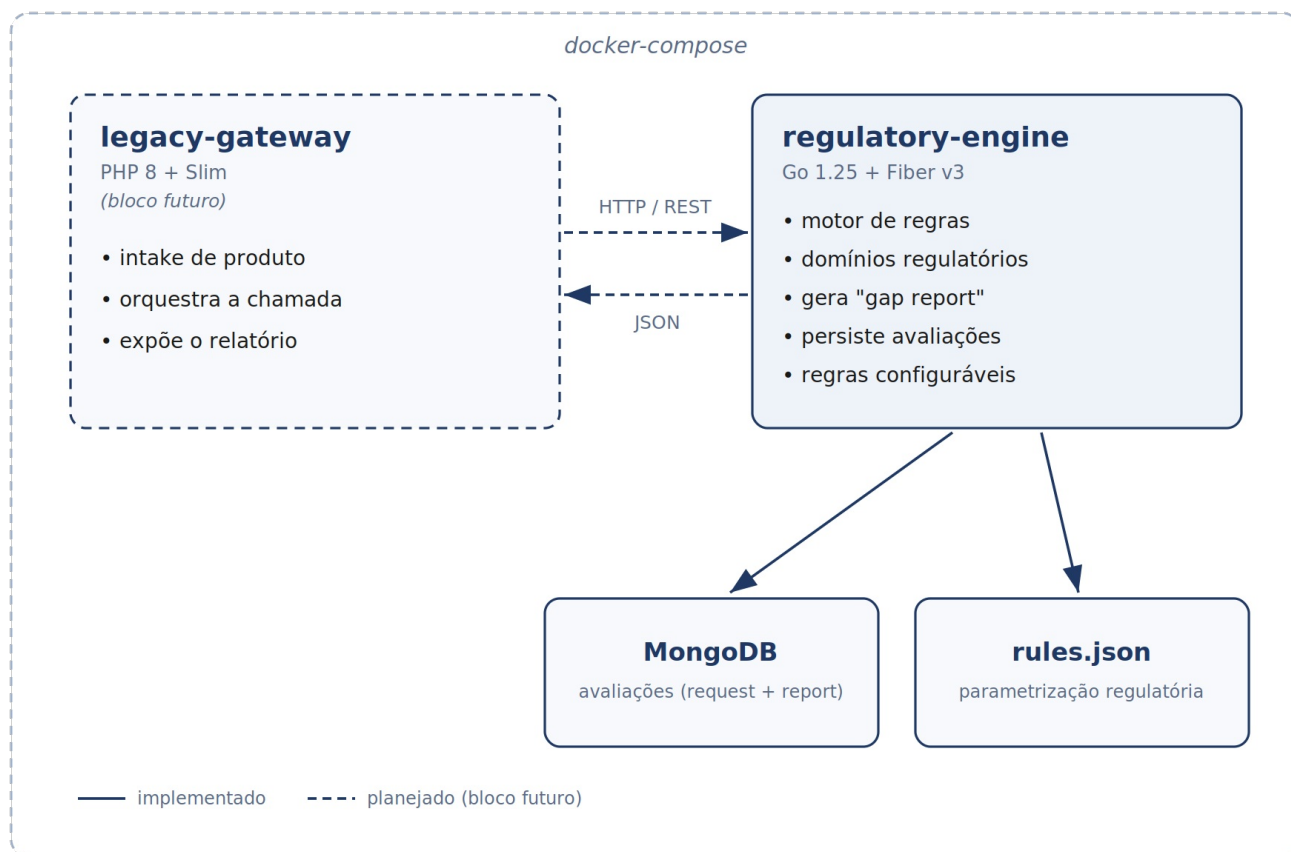
Fazer essa adequação manualmente, produto a produto, é lento e propenso a erro. Este projeto sistematiza a verificação: um serviço central que, dada uma operação, aponta objetivamente as exigências regulatórias brasileiras aplicáveis e as adaptações necessárias.

O projeto demonstra três capacidades:

1. **Entendimento do domínio** financeiro/regulatório brasileiro.
2. **Transformar regra de negócio em software** bem estruturado.
3. **Arquitetura poliglota** com integração entre um sistema legado e uma nova API.

3. Arquitetura

Dois microsserviços containerizados, orquestrados por Docker Compose, reproduzindo um padrão real de migração/modernização gradual — a coexistência entre um sistema legado e uma nova API. Hoje o motor e o MongoDB já rodam no compose; o gateway entra em bloco futuro:



Papéis:

- **regulatory-engine (Go + Fiber):** coração do projeto. Recebe uma operação/produto, devolve o veredito de conformidade e **persiste cada avaliação no MongoDB** (requisição + veredito, como registro de auditoria), expondo o histórico por API. Contém o motor de regras e os domínios regulatórios e lê a parametrização de um arquivo de regras. Não guarda estado em memória entre requisições — todo estado vive no banco —, então escala horizontalmente.
- **legacy-gateway (PHP + Slim, bloco futuro):** representa o sistema legado de uma instituição financeira. Faz o *intake* das submissões e chama o motor Go. É o ponto de integração legado ↔ moderno; o desenho detalhado (por exemplo, como consome o histórico) será definido no bloco do gateway.

Decisão de arquitetura-chave: as regras (limites, alíquotas, gatilhos de reporte) **não são hardcoded** — vivem em parametrização configurável (`rules.json` , evoluível para tabela no banco). É uma boa prática essencial no contexto regulatório, porque norma muda com frequência.

Validação da parametrização (fail-fast). Por ser editável sem recompilar, a parametrização também pode ser editada *errada*. Por isso `rules.Load()` valida o arquivo logo após interpretá-lo (`Parameters.Validate()`) e, se algo estiver incoerente com o que as regras assumem, o serviço **não sobe** — mesmo princípio já usado na conexão com o MongoDB. Os problemas são acumulados (`errors.Join`) para o erro de boot listar tudo o que precisa ser corrigido de uma vez. Checagens atuais:

- `pld.reporting_threshold.currency` deve ser `BRL` : o teto é comparado com o valor da operação já convertido para reais; outra moeda seria tratada como BRL sem aviso.
- `fx.usd_brl` deve ser maior que 0: câmbio zero zeraria toda conversão (IOF de R\$ 0,00 e nenhuma operação atingindo o teto).
- `pld.reporting_threshold.amount` deve ser maior que 0.
- `iof.international_transfer.rate` deve ser maior que 0 e menor que 1 (100%).

O critério "maior que zero" tem um segundo papel: no JSON, **chave ausente vira zero** (o `decimal` não distingue "não veio" de "veio 0"), então uma chave esquecida ou digitada errado também é barrada. Por isso a alíquota zero não é aceita mesmo existindo operações com IOF 0% na norma: aceitar zero aceitaria esquecimento, e a regra emitiria `adaptation_required` com IOF de R\$ 0,00 — veredito incoerente. Alíquota zero real se representa desligando a regra. O limite superior (`< 1`) é só sanidade; valores plausíveis porém errados (ex.: `0.38` no lugar de `0.0038`) exigiriam um teto de domínio, deixado de fora de propósito.

Um teste carrega o `config/rules.json` versionado, de modo que uma parametrização inválida quebra o CI antes de chegar ao boot.

4. Domínios regulatórios (escopo inicial)

Sobre os valores: os números abaixo são *ilustrativos* para a estrutura funcionar. Antes de finalizar, confirmar os parâmetros vigentes nas fontes oficiais (Bacen, COAF/UIF, Receita Federal) e ajustar no `rules.json`. A arquitetura configurável existe justamente para isso.

Cada item indica se já está **implementado** (com o código da regra) ou **planejado**.

4.1. PLD/FT + COAF (Prevenção à Lavagem de Dinheiro)

- Operações com valor, convertido para BRL, igual ou acima do teto de comunicação → **reportáveis ao COAF**. *Implementado* (`pld.reporting_threshold`).
- Screening contra lista de **PEP / sancionados** (lista mock local). *Implementado* (`pld.pep_screening`).
- Outras hipóteses de comunicação (ex.: espécie acima de teto, fracionamento suspeito). *Planejado*.
- Saída: alertas + indicação de comunicação obrigatória.

4.2. Câmbio / IOF (o core da "tropicalização")

- Cálculo de **IOF** na transferência internacional EUA → BR. *Implementado* (`iof.international_transfer`).
- Conversão cambial com taxa parametrizável — hoje apenas USD → BRL. *Implementado*.
- Regras para outros tipos de operação (investimento, empréstimo). *Planejado*.
- Saída: tributos aplicáveis + adaptações necessárias vs. o produto original.

4.3. (Roadmap) Limites Bacen/Pix + LGPD

- Fica documentado como próxima fase, para o projeto ter história de evolução.

5. Contrato da API (regulatory-engine)

```
POST /api/v1/evaluate
→ avalia um produto/operação, PERSISTE e retorna a avaliação criada (201)
{ id, created_at, rules_version, request, report }
GET /api/v1/evaluations
→ lista as avaliações mais recentes
GET /api/v1/evaluations/{id}
→ recupera uma avaliação pelo id
GET /api/v1/health
→ liveness: processo no ar (não consulta o banco)
GET /api/v1/ready
→ readiness: 200 se o MongoDB responde ao ping, 503 se não
```

Exemplo de requisição (`POST /api/v1/evaluate`):

```
{
  "product": {
    "type": "personal_loan",
    "origin": "US",
    "origin_currency": "USD"
  },
  "operation": {
    "amount": 75000.00,
    "currency": "USD",
    "method": "international_transfer",
    "counterparty": { "name": "John Doe", "pep": false }
  }
}
```

Exemplo do report (dentro da avaliação retornada pelo POST):

```
{
  "compliant": false,
  "results": [
    {
      "domain": "IOF",
      "status": "adaptation_required",
      "detail": "Entrada convertida de USD para BRL a 5.00; IOF de 0.38% aplicável (produto original não previa tributação).",
      "calculated_amount": "1425.00",
      "reference": "rule:iof.international_transfer"
    },
    {
      "domain": "PLD_COAF",
      "status": "reportable",
      "detail": "Valor acima do teto de comunicação automática ao COAF.",
      "reference": "rule:pld.reporting_threshold"
    }
  ],
  "required_adaptations": [
    "Incluir cálculo e retenção de IOF no fluxo de entrada.",
    "Gerar comunicação ao COAF para a operação."
  ]
}
```

Valores monetários trafegam como **string com 2 casas** ("1425.00"), para não perder precisão no cliente. Na entrada, o campo `amount` aceita número ou string. Alíquotas e câmbio (parametrização) usam decimal simples.

Validação de entrada. Antes de chegar ao motor, a requisição é validada por `EvaluationRequest.Validate()` (camada de domínio). Sem essa barreira, campos ausentes viram *zero values* do Go (valor 0, método vazio, nome vazio): as regras "não se aplicam" e a operação sairia **conforme por omissão** — o falso negativo silencioso que um motor de compliance não pode produzir.

Campo	Regra
<code>product.type</code>	valor conhecido (<code>personal_loan</code>)
<code>product.origin</code>	código de país ISO 3166-1 alpha-2 (ex.: <code>US</code>)
<code>product.origin_currency</code>	código de moeda ISO 4217 (ex.: <code>USD</code>); informativo
<code>operation.amount</code>	maior que zero, no máximo 2 casas decimais
<code>operation.currency</code>	código de moeda ISO 4217 e, por ora, somente <code>USD</code>
<code>operation.method</code>	valor conhecido (<code>international_transfer</code>)
<code>operation.counterparty.name</code>	obrigatório (sem ele o screening fica cego)

Todos os campos inválidos são reportados de uma vez, com `400 Bad Request`, e nada é persistido:

```
{
  "error": "requisicao invalida",
  "details": [
    { "field": "operation.amount", "message": "deve ser maior que zero" },
    { "field": "operation.method", "message": "metodo de operacao desconhecido" }
  ]
}
```

Decisões: a validação fica no domínio (não em tags de biblioteca) para manter a lista de valores válidos junto das constantes e testável sem HTTP; a origem é obrigatória porque a avaliação é um registro de auditoria, e tornar um campo obrigatório depois quebraria clientes, enquanto afrouxar não. Para os códigos de país e moeda valida-se só o formato, sem consultar a lista oficial.

Moedas. Os dois campos de moeda têm papéis diferentes:

- `operation.currency` é a moeda do `amount` e **entra no cálculo**: IOF e teto do COAF convertem o valor para

BRL. Como a parametrização só tem o câmbio `usd_brl`, apenas `USD` é aceito; qualquer outra moeda retorna `400` com `"moeda nao suportada (aceita: USD)"`. Recusar é preferível a calcular errado — antes desta restrição, `"BRL"` era tratado como dólar sem aviso.

- `product.origin_currency` **descreve o produto de origem** e não entra em nenhum cálculo. É obrigatório (registro de auditoria completo) e tem o formato validado, mas não é restrito a `USD`: restringir um campo que não afeta o resultado rejeitaria requisições sem motivo técnico. Passa a ter validação semântica quando alguma regra depender dele.

Suportar só `USD` é **decisão de escopo**, não limitação acidental: o cenário do projeto é a tropicalização `EUA` → `Brasil`. Aceitar outra moeda exigiria trocar `fx.usd_brl` por uma tabela de câmbio por moeda e um conversor único usado pelas regras, com a lista de moedas aceitas derivada da parametrização.

6. Estrutura de pastas (monorepo)

```
regulatory-compliance-engine/
├── .github/workflows/ci.yml      # CI: gofmt, build, vet, test + docker build
├── docker-compose.yml          # MongoDB + motor
├── Makefile                     # atalhos de desenvolvimento
├── README.md
├── docs/
│   ├── especificacao_tecnica.md
│   ├── cenarios_de_teste.md    # catálogo de cenários (entrada → resultado esperado)
│   ├── cenarios_de_teste.http  # os mesmos cenários, executáveis (REST Client)
│   └── arquitetura.png         # diagrama (fonte: arquitetura.svg)
├── regulatory-engine/          # Go + Fiber
│   ├── cmd/api/main.go         # ponto de entrada (montagem da aplicação)
│   ├── internal/
│   │   ├── config/            # configuração via ambiente
│   │   ├── database/          # conexão com o MongoDB
│   │   ├── model/              # structs do domínio (contrato) + validação de entrada
│   │   ├── money/              # tipo monetário (decimal, sempre 2 casas)
│   │   ├── engine/            # interface Rule + avaliador
│   │   ├── rules/              # regras (IOF, PLD) + carregamento e validação do rules.json
│   │   ├── repository/        # persistência das avaliações + índices
│   │   └── httpapi/           # servidor, rotas e handlers
│   ├── config/rules.json       # parametrização regulatória
│   ├── go.mod
│   └── Dockerfile              # imagem multi-stage (distroless)
└── legacy-gateway/            # PHP + Slim (bloco futuro; estrutura definida no bloco)
```

Os índices do MongoDB são criados pelo próprio motor no boot (ver seção 7), sem script de *seed* separado.

7. Modelo de dados (MongoDB)

Banco `regulatory`. A avaliação é gravada como **um único documento embedded** na coleção `evaluations` — o modelo idiomático de MongoDB: uma escrita, e uma leitura traz o quadro completo, sem `"join"`.

- **evaluations**: `{ _id, created_at, rules_version, request { ... }, report { ... } }` — a submissão original e o veredito produzido, juntos no mesmo documento, com a versão da parametrização que o produziu.

Versão da parametrização (`rules_version`). Um registro de auditoria precisa responder não só *o que* foi decidido, mas *sob quais regras*: se a alíquota de IOF mudar no `rules.json`, um `calculated_amount` antigo só continua explicável se o registro disser qual arquivo estava em vigor. Por isso cada avaliação grava `rules_version = "sha256:" + hash dos bytes do rules.json`, calculado no `rules.Load` e também registrado no log de boot.

- **Por que o hash dos bytes**: qualquer um confere com ferramenta padrão (`sha256sum config/rules.json`), sem depender do código; e o hash não "esquece" de mudar, ao contrário de um número de versão escrito à mão. A normalização de fim de linha para LF (`.gitattributes`) garante o mesmo hash no Windows, no Linux e na imagem Docker.
- **Por que fora do report**: é proveniência do registro (como `created_at`), não resultado de regra; o motor continua sem conhecer a versão, que é injetada no servidor.
- **Avaliações anteriores** a este campo ficam sem ele: não há como saber retroativamente qual arquivo estava em uso, então não há migração.

Índices. `created_at_desc ( { created_at: -1 } )` atende a listagem das avaliações mais recentes: sem ele, a

consulta faria *collection scan* com ordenação em memória. O índice é declarado pelo próprio repositório (`EnsureIndexes`) e criado no boot, com fail-fast — quem consulta declara o índice de que precisa, e ele existe em qualquer ambiente, não só no compose. Como o `createIndexes` do MongoDB é idempotente, rodar a cada boot não tem custo. Em coleções grandes, a criação migraria para um passo de migração separado, para não alongar o boot.

Identificador (`_id` como `string`). O id é gerado como `ObjectId` (`primitive.NewObjectId()`), mas gravado como a sua representação hexadecimal em **`string`**, e não com o tipo nativo `ObjectId`. É um *tradeoff* consciente:

- **Ganho:** o domínio fica livre de tipos do driver (`model.Evaluation.ID` é `string`, sem importar `primitive`), o JSON expõe o id sem serialização customizada, e o handler repassa o parâmetro da rota direto à consulta — um id malformatado simplesmente não encontra nada (`404`), sem etapa de conversão.
- **Custo:** a chave ocupa 24 bytes em vez de 12, o que aumenta o índice `_id`, e perde-se o timestamp embutido do `ObjectId` (`getTimestamp()`). O segundo custo é neutralizado pelo campo explícito `created_at`; a ordem cronológica também se preserva, porque o hexadecimal de tamanho fixo ordena igual ao `ObjectId`.
- **Reversão:** migrar para o tipo nativo depois exigiria reescrever o `_id` de todos os documentos — por isso a decisão fica registrada aqui. No volume atual, a simplicidade compensa o custo de armazenamento.

Valores monetários são gravados como **Decimal128** (o decimal nativo do Mongo), preservando a precisão e permitindo consultas e agregações por valor.

Decisão de modelagem (embedded vs. referência). `request` e `report` têm relação 1:1, nascem juntos e são sempre lidos juntos — portanto ficam **embutidos** no mesmo documento (separá-los em duas coleções seria um anti-padrão: duas escritas não-atômicas e um *join* na leitura, sem benefício). Como a avaliação é um **registro de auditoria**, o documento é tratado como um **retrato imutável** do que foi avaliado e do veredito daquele momento. Promover a **contraparte** a entidade própria (coleção `parties`) só se justificaria com uma **identidade estável** — um documento (CPF/CNPJ), não o nome —, o que exigiria *entity resolution* (casamento aproximado contra listas de sanção), um problema à parte e fora do escopo deste projeto.

8. Testes e qualidade

A garantia de qualidade do `regulatory-engine` se apoia em três pilares implementados: uma suíte de testes automatizados, um pipeline de integração contínua que a executa a cada mudança, e logs estruturados para observabilidade.

8.1. Estratégia de testes

A suíte usa `testify` e cobre as três camadas do motor de forma independente:

- **Testes unitários (table-driven).** As regras de negócio — cálculo de IOF, teto de comunicação ao COAF, screening de PEP/sancionados —, a validação de entrada e o tipo monetário são testados com tabelas de casos (entrada esperada vs. saída), o padrão idiomático em Go. Cada regra é verificada isoladamente: quando se aplica, o valor calculado, e quando **não** se aplica (retorno nulo).
- **Teste de agregação do motor com dublê.** O avaliador (`engine`) é testado contra uma **regra falsa** (`fakeRule`) controlada pelo teste, não contra as regras reais. Assim se verifica apenas a responsabilidade do motor — agregar resultados, ignorar regras que não se aplicam, decidir conformidade e propagar erro — sem acoplamento ao comportamento de IOF ou PLD.
- **Teste de handler HTTP sem banco.** Os handlers dependem da **interface** `EvaluationStore`, não do repositório concreto. Nos testes, injeta-se um **store falso em memória** (`fakeStore`) e exercitam-se as rotas de ponta a ponta com `app.Test` (sem abrir porta de rede nem exigir MongoDB): `POST /evaluate` retornando `201` e persistindo, corpo malformatado ou requisição incompleta retornando `400` (com a lista de campos inválidos e sem persistir), e busca inexistente retornando `404`. Um teste adicional verifica, nas três rotas que acessam o banco, que o contexto recebido pelo store **deriva do contexto da requisição** (um valor anexado por *middleware* chega ao store) e carrega o timeout do handler. O readiness é testado com um `Pinger` falso (banco acessível → `200`, indisponível → `503`), e um teste garante que o liveness continua `200` com o banco fora. O subcomando `healthcheck` é testado contra servidores HTTP de teste (`httptest`).
- **Cenários regulatórios de aceitação.** Os testes acima isolam cada peça; nenhum deles junta as regras reais **com o `config/rules.json` versionado**. O teste de aceitação faz isso: monta o motor com a mesma função usada em produção (`rules.NewEngine`, único ponto de montagem — o `main` não monta regras por conta própria, então teste e produção não divergem) e verifica, para dez cenários (C01-C10), o veredito completo: domínios, status, valores calculados e detalhes. Inclui as fronteiras que mais escondem erro — arredondamento do IOF e um centavo abaixo / exatamente no teto do COAF. Se alguém alterar a

parametrização, o teste falha e aponta quais vereditos mudaram (verificado alterando a alíquota: os dez cenários falham). Os mesmos cenários estão em `cenarios_de_teste.md` e `cenarios_de_teste.http`, para execução manual contra a API.

- **Integração do repositório com MongoDB real.** Os dublês garantem que os handlers chamam o store corretamente, mas não que o MongoDB grava e devolve o que o código espera. Os testes do pacote `repository` rodam contra um Mongo de verdade e cobrem: ida e volta de uma avaliação completa (Decimal128, resultado sem valor calculado, `rules_version`), `FindByID` devolvendo `nil` para id inexistente, `List` ordenado do mais recente e limitado, e `EnsureIndexes` idempotente. Rodam só quando `MONGO_TEST_URI` está definida — no CI, por um *service container*; localmente, com `make test-integration` — e se pulam caso contrário, para o `go test` comum continuar sem Docker. Cada teste usa um banco de nome único, apagado ao final. Optou-se por essa forma em vez de *testcontainers*: mesma cobertura, sem dependência nova.

O teste de ida e volta revelou uma inconsistência real: o `created_at` da resposta do `POST` tinha precisão de nanossegundos (relógio do Go), mas o MongoDB armazena milissegundos, então um `GET` do mesmo recurso devolvia outro valor. O repositório passou a truncar a data em milissegundos antes de gravar.

O ponto de projeto que torna isso possível é a **injeção de dependência** adotada nos blocos anteriores: como o servidor recebe o motor e o store por interface, ambos podem ser substituídos por dublês nos testes. Testes rápidos, determinísticos e que rodam em qualquer máquina limpa — inclusive no CI, sem infraestrutura.

8.2. Integração contínua (CI)

Um workflow de **GitHub Actions** roda a cada `push` na `main` e em todo *pull request*, numa máquina limpa do runner. A sequência reproduz a verificação local:

1. `gofmt` — falha o build se houver código fora do padrão de formatação da linguagem.
2. `go build ./...` — garante que todo o módulo compila.
3. `go vet ./...` — análise estática de problemas comuns.
4. `go test ./...` — executa a suíte descrita acima, incluindo os testes de integração do repositório: o job sobe um MongoDB 7 como *service container* e define `MONGO_TEST_URI`.

Em paralelo, um segundo job constrói a imagem Docker do motor (`docker build`), para que uma quebra no Dockerfile seja detectada no mesmo push, e não só no momento do deploy.

O ambiente é fixado em Go 1.25 com cache de módulos. O valor concreto: a verificação deixa de depender da disciplina manual do desenvolvedor — um arquivo esquecido no commit, um `go.sum` inconsistente ou código desformatado são barrados antes de entrar na `main`. O estado do pipeline é exposto por um *badge* no README.

8.3. Observabilidade

Os dois serviços emitem **logs estruturados em JSON** (via `slog` no motor Go). Um *middleware* de requisição registra método, rota, status e latência de cada chamada em formato de campo — pronto para ser filtrado por um agregador (Loki, ELK, CloudWatch) sem *parsing* de texto livre.

Propagação de contexto. Os handlers derivam o contexto das chamadas ao MongoDB do contexto da requisição (`c.Context()`), com timeout de 5s, em vez de partir de `context.Background()`. A derivação fica num helper único (`newRequestContext`, com o prazo na constante `storeTimeout`), para que nenhum handler novo parta do contexto errado nem repita o prazo. O ganho é um ponto único de propagação: quando entrar um *middleware* de request-id, tracing (OpenTelemetry) ou prazo por requisição, o que ele anexar chega ao banco sem alterar os handlers, e os *spans* do Mongo ficam ligados à requisição que os originou.

Limitação conhecida: isso **não** cancela a query quando o cliente desconecta. O Fiber roda sobre o fasthttp, que por desempenho não sinaliza desconexão durante o handler (o `Done()` da requisição só fecha no desligamento do servidor). Cancelamento por desconexão exigiria um servidor baseado em `net/http`, sem demanda que justifique hoje; o timeout de 5s é o limite efetivo.

Liveness e readiness. São duas perguntas diferentes, com dois endpoints:

- `GET /api/v1/health` (**liveness**) responde se o processo está vivo e **não consulta o banco**. Quem reage a uma falha de liveness reinicia o processo — e reiniciar não conserta um banco fora do ar, só geraria um ciclo de reinícios.
- `GET /api/v1/ready` (**readiness**) pinga o MongoDB (prazo de 2s) e responde `503` se ele não responder. Falhar aqui tira o serviço do tráfego sem reiniciá-lo.

No compose, o motor tem `healthcheck` baseado no `readiness` — é o que permitirá ao gateway declarar `depends_on: condition: service_healthy`. Como a imagem `distroless` não tem `curl` nem `wget`, o próprio binário oferece o subcomando `regulatory-engine healthcheck`, que consulta o `/ready` local e sai com `0` ou `1`.

Encerramento gracioso. Ao receber `SIGTERM` (enviado por `docker stop` e orquestradores) ou `SIGINT`, o motor para de aceitar conexões, espera as requisições em andamento terminarem (até 10s, `shutdownTimeout`) e só então desconecta o MongoDB. Dois cuidados:

- O `Listen` roda numa goroutine e o encerramento usa `ShutdownWithTimeout`, que **bloqueia** até as requisições terminarem. O `GracefulContext` do Fiber não foi usado porque nele o `Listen` retorna assim que o listener fecha, antes das requisições em andamento — o banco seria desconectado no meio delas.
- A janela entre `SIGTERM` e `SIGKILL` precisa caber o **pior caso** do encerramento, senão o orquestrador mata o processo antes: 10s esperando requisições + 5s de prazo para desconectar o MongoDB = 15s. O compose define `stop_grace_period: 20s` (o padrão do Docker, 10s, não caberia). O pior caso foi observado na prática: com o Mongo fora do ar, o `Disconnect` consome o prazo inteiro.

Verificação manual (Docker): com o Mongo de pé, `docker stop` encerra em menos de 1s; com uma requisição em andamento, ela recebe a resposta completa antes do processo sair, e o container termina com código `0` (e não `137`, de `SIGKILL`).

8.4. Demonstração

- **README** com contexto de negócio, diagrama, como rodar (`docker compose up`, sem Go instalado), exemplos de chamada em `curl` e o **roadmap**.

9. Roadmap (evolução futura)

1. **Gateway PHP + Slim:** serviço que representa o sistema legado e chama o motor Go (integração legado ↔ novo).
2. Domínios adicionais: Limites Bacen/Pix, LGPD, SCR.
3. **Regras em banco com vigência:** hoje cada avaliação já registra o hash da parametrização usada (`rules_version`, seção 7); a evolução é manter o histórico de versões com datas de vigência, para avaliar uma operação pelas regras válidas na data dela.